



Contents

Contents	2
I. Gamma Containment Protocol	3
II. Prólogo	4
III. Introducción	5
A. Ejercicio 0: La Cámara de Contención	6
B. Ejercicio 1: Cuando Todo Sale Verde	7
C. Ejercicio 2: Múltiples Cámaras (orden de cierre)	8
D. Ejercicio 3: El Reactor que No Quiere Cerrar	9
E. Ejercicio 4: El Guardián de Recursos (patrón try-finally manual)	10
F. Ejercicio 5: Comparativa Final (Reflexión)	11
Reto. "Área de Plantación, pero con recursos bien cerrados"	12
Ejecución	14

I. Gamma Containment Protocol

Ya conoces try/catch/finally (módulo 02) y ya usaste try-with-resources de pasada (módulo 04). Ahora vas a entender **por qué** existe esa magia y vas a construir tus propios recursos gestionables — literalmente, cámaras de contención que se cierran solas.

Instrucciones Generales

- ✓ **Versión:** Java 17+ (OpenJDK).
- ✓ **Estructura:** Una clase por archivo (Puedes usar package).
- ✓ **Linter Obligatorio:** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ **Comentarios:** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ **IDE:** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ **Compilación:** Vía terminal: `javac Clase.java` y `java Clase`.

Entrega y Evaluación

- Entrega en repositorio Git.

II. Prólogo

Bruce Banner aprendió una regla fundamental en el laboratorio: **todo lo que se abre debe poder cerrarse.**

Cada experimento con gamma-radiación depende de un sistema de contención perfectamente controlado. Una cámara abierta, una válvula sin asegurar o un reactor que no se apaga correctamente no son simples errores de programación: son potenciales incidentes de nivel Hulk. En programación ocurre algo similar.

Cada vez que abres un archivo, estableces una conexión con una base de datos, creas un flujo de comunicación o reservas un recurso del sistema, estás asumiendo una responsabilidad:

"Este recurso será liberado correctamente, sin importar lo que ocurra durante el proceso."

El problema es que los errores suceden. Las excepciones aparecen, los datos fallan, las conexiones se interrumpen y los experimentos pueden salirse de control. Por eso Java proporciona mecanismos para garantizar que los recursos se cierren de forma segura incluso cuando el código falla.

En el módulo anterior aprendiste a trabajar con datos. Ahora aprenderás a trabajar con la otra cara de la programación profesional: **la gestión responsable de los recursos.** Porque en este laboratorio olvidar cerrar algo no es un simple descuido.

Es una fuga de contención.



III. Introducción

¡Bienvenido al Módulo 05: Gamma Containment Protocol!

Hasta ahora ya has trabajado con bloques **try/catch/finally** y has visto brevemente **try-with-resources**. En este módulo descubrirás qué problema resuelve realmente esta característica de Java y por qué se convirtió en una herramienta esencial para escribir código seguro y mantenible.

Aprenderás a:

- Entender por qué los recursos deben cerrarse siempre.
- Dominar el funcionamiento de **try-with-resources**.
- Crear tus propios recursos gestionables.
- Implementar la interfaz **AutoCloseable**.
- Diseñar componentes que limpien su propio estado automáticamente.
- Evitar fugas de memoria, conexiones abiertas y recursos bloqueados.

La misión es sencilla: **Abrir. Usar. Cerrar. Siempre.**

En el laboratorio de Bruce Banner, una cámara de contención no espera a que recuerdes apagarla. En Java, un recurso bien diseñado tampoco debería depender de la memoria del programador. Tu objetivo será construir sistemas capaces de protegerse incluso cuando las cosas se vuelvan... verdes.



A. Ejercicio 0: La Cámara de Contención

- ✓ **Directorio:** ex0/
- ✓ **Archivo:** ContainmentChamber.java, Demo.java
- ✓ **Funciones Autorizadas:** AutoCloseable, System.out.println, @Override
- ✓ **Método:** Any.
- ✓ **Restricción:**
 - ✓ Debes crear una clase que implemente **AutoCloseable**.
 - ✓ El método close() **no debe ser llamado manualmente**. Java debe encargarse de cerrar la cámara automáticamente al abandonar el bloque try, incluso si dentro del bloque ocurriera una excepción.

Objetivo

Bruce Banner no puede permitirse depender de que alguien recuerde cerrar manualmente una cámara de contención. En este ejercicio crearás un recurso que se gestione automáticamente mediante el mecanismo **try-with-resources** de Java. Tu misión es construir una clase llamada **ContainmentChamber** que represente una cámara de contención gamma.

La clase debe:

1. Implementar la interfaz **AutoCloseable** con **ContainmentChamber.java**.
2. Definir el método **close()**.
3. Hacer que **close()** imprima exactamente: **Chamber sealed. Gamma levels stable**.

Salida Esperada

```
Opening containment chamber...
Running gamma experiment...
Chamber sealed. Gamma levels stable.
```

B. Ejercicio 1: Cuando Todo Sale Verde

- ✓ **Directorio:** ex1/
- ✓ **Archivo:** HulkOutbreak.java, Demo.java
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** AutoCloseable, try, RuntimeException, @Override
- ✓ **Restricción:**
 - El método `close()` debe ejecutarse **siempre**, incluso cuando el experimento falle dentro del bloque `try`.
 - No usar el `catch`.

Objetivo

En el laboratorio de Bruce Banner, los fallos no avisan. Un experimento puede funcionar perfectamente durante unos segundos y, de repente, alcanzar niveles críticos de radiación gamma. Tu misión es simular un experimento que falla durante su ejecución y demostrar que Java mantiene el protocolo de seguridad activo gracias a **try-with-resources**.

Debes crear un recurso de contención que:

1. Se abra correctamente.
2. Ejecute un experimento gamma.
3. Lance una **RuntimeException** con el mensaje: **Gamma overload detected!**
4. Cierre la cámara automáticamente, aunque el experimento haya fallado.

Salida Esperada

```
Opening containment chamber...  
  
ERROR: Gamma detected!  
Chamber sealed. Gamma levels stable.  
Exception in thread "main" java.lang.RuntimeException: Gamma specific  
overload detected!  
    at ex1.HulkOutbreak.Work(HulkOutbreak.java:9)  
    at ex1.Demo.main(Demo.java:6)
```

C. Ejercicio 2: Múltiples Cámaras (orden de cierre)

- ✓ **Directorio:** ex2/
- ✓ **Archivo:** MultiChamberLab.java, ContainmentChamber.java
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** AutoCloseable, try-with-resources, @Override
- ✓ **Restricción:** *Debes abrir al menos **dos recursos distintos que implementen AutoCloseable** dentro de una única declaración try.*

Objetivo - El laboratorio de Bruce Banner cuenta con varias cámaras de contención conectadas. Cuando un experimento utiliza más de un recurso, Java debe garantizar que todos sean cerrados correctamente.

En este ejercicio crearás una situación con dos cámaras:

- ContainmentChamber A
- ContainmentChamber B

Requisitos

El programa debe:

- Crear dos recursos diferentes que implementen AutoCloseable.
- Abrir ambos en la misma declaración try-with-resources.
- Mostrar el mensaje de apertura de cada cámara.
- Implementar close() para mostrar el cierre correspondiente.
- Permitir que Java gestione automáticamente el orden de cierre.
- No llamar manualmente a close().

Salida Esperada

```
Opening Chamber A...
Opening Chamber B...
Closing Chamber B...
Closing Chamber A...
```


D. Ejercicio 3: El Reactor que No Quiere Cerrar

- ✓ **Directorio:** ex3/
- ✓ **Archivo:** StubbornReactor.java, ReactorMalfunctionException.java, Demo.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** AutoCloseable, throw, excepciones dentro de close().
- ✓ **Restricción:** El método close() debe poder lanzar una excepción propia llamada **ReactorMalfunctionException**, y dicha excepción debe ser gestionada correctamente desde el código principal.

Objetivo

Debes crear un recurso StubbornReactor que implemente AutoCloseable.

El reactor debe

1. Tener un nivel de energía (energyLevel).
2. Intentar sellarse cuando Java ejecute automáticamente close().
3. Lanzar una excepción personalizada ReactorMalfunctionException si el nivel de energía supera un límite crítico (100).

Requisitos

El programa debe

- Crear una excepción propia llamada ReactorMalfunctionException.
- Hacer que close() pueda lanzar esa excepción.
- Utilizar try-with-resources.
- Permitir que Java invoque close() automáticamente.
- Capturar la excepción correctamente desde el código principal.
- Evitar que el programa termine de forma incontrolada.
- No debes llamar manualmente a close().

Salida Esperada

```
Running gamma experiment...
Attempting to seal reactor...
Reactor refused to close! Energy level: 150
```

E. Ejercicio 4: El Guardián de Recursos (patrón try-finally manual)

- ✓ **Directorio:** ex4/
- ✓ **Archivo:** ManualGuardian.java, Connection.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** try/catch/finally
- ✓ **Restricción:**
 - **No está permitido utilizar try-with-resources ni AutoCloseable.** Debes gestionar el cierre del recurso manualmente mediante un bloque finally.

Objetivo– Antes de Java 7, los recursos no podían cerrarse automáticamente. Cada archivo, conexión o flujo abierto debía cerrarse de forma explícita por el programador.

El problema era evidente: si durante la ejecución ocurría una excepción, era muy fácil olvidar liberar el recurso, provocando fugas de memoria, conexiones bloqueadas o archivos que permanecían abiertos. En este ejercicio viajarás al pasado y escribirás el mismo patrón que los desarrolladores utilizaban durante años.

El programa debe:

1. Abrir una conexión.
2. Comenzar una operación.
3. Simular un fallo durante la ejecución.
4. Ejecutar el bloque **finally**.
5. Cerrar la conexión manualmente.

Aunque el experimento falle, el protocolo de cierre debe ejecutarse siempre.

Salida Esperada

```
Connecting to Banner's lab...  
  
ERROR: Connection unstable!  
[finally] Closing connection manually.
```

F. Ejercicio 5: Comparativa Final (Reflexión)

- ✓ **Directorio:** ex5/
- ✓ **Archivo:** ContainmentReport.java

Objetivo

Has llegado al final del protocolo de contención del laboratorio de Bruce Banner. A lo largo de este módulo has aprendido dos formas distintas de gestionar recursos:

- El enfoque tradicional mediante ***try-finally***.
- El enfoque moderno mediante ***try-with-resources*** y ***AutoCloseable***.

En este ejercicio elaborarás un pequeño informe que resuma lo aprendido y explique por qué Java introdujo este nuevo mecanismo.

Puedes escribir el informe como

- Un comentario Javadoc extenso dentro de ContainmentReport.java.
- Un bloque de comentarios tradicional (*/* ... */*).
- Un pequeño documento explicativo en el propio archivo.

No es necesario escribir código ejecutable.

Reto. "Área de Plantación, pero con recursos bien cerrados"

- **Directorio:** reto/, solo un archivo.
- **Dónde ejecutarlo:** en el mismo proyecto *hola-bosque* que ya tienes no crees uno nuevo.

Objetivo: en el módulo 05 aprendiste que **AutoCloseable** garantiza que un recurso se cierre pase lo que pase. Ahora vas a simular que cada vez que alguien calcula un área en tu navegador, se abre un "sensor de medición" en el vivero — y ese sensor debe cerrarse siempre, incluso si el cálculo falla.

Qué vas a agregar dentro de tu clase `HolaBosqueApplication.java`, justo debajo del método `areaSegura()` (el del Reto 02):

Primero, una clase interna (***MeasurementSensor***) que simule el sensor:

```
static class MeasurementSensor implements AutoCloseable {
    public MeasurementSensor() {
        System.out.println("Sensor activado.");
    }

    @Override
    public void close() {
        System.out.println("Sensor apagado correctamente.");
    }
}
```

Y el nuevo método en ***HolaBosqueApplication.java***:

No necesitas **import** nuevo — reutilizas los mismos que ya tenías (`@RequestParam`, `@GetMapping`).

Cómo probarlo

1. Guarda, reinicia el servidor.
2. Ve a esta URL en tu navegador:
`http://localhost:8080/area-segura?longitud=10&ancho=5`
3. Deberías ver: El área de la sección de reforestación es: 50.0 m²
4. Ahora prueba con un valor negativo:
`http://localhost:8080/area-segura?longitud=-10&ancho=5`
5. En vez de romperse, deberías ver: Error: Longitud y ancho deben ser positivos.

Qué está pasando (sin entrar en detalle todavía)

- El **try/catch** que usaste en consola (módulo 02) funciona exactamente igual dentro de un método web.
- La diferencia es que en consola hacías **System.out.println(e.getMessage())**, y aquí haces **return "Error: " + e.getMessage()** — en vez de imprimir, devuelves el mensaje como respuesta de la página.
- El servidor **sigue vivo** después del error, igual que tu programa de consola seguía funcionando después de capturar la excepción.

La comparación clave para que se te quede

Java puro (consola, módulo 02)	Spring Boot (web, reto 02)
try {...} catch (IllegalArgumentException e)	Exactamente el mismo bloque, dentro del método del controlador
System.out.println(e.getMessage())	return "Error: " + e.getMessage()
El programa sigue pidiendo datos tras el error	La API sigue respondiendo a otras peticiones tras el error

Ejecución – Si no te funciona el botón de IntelliJ

```
cd "C:\Users\XXXX\Documents\hola-bosque\hola-bosque"
.\mvnw.cmd spring-boot:run
```

Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando `javac` te dirá "**archivo no encontrado**" y te entrarán ganas de tirar el PC por la ventana.

- ✓ `cd Desktop/proyectos_java/ex00`
- ✓ `javac FtHelloForest.java` -> La Compilación (El traductor)
- ✓ `java FtHelloForest` -> La Ejecución (El lanzamiento)